



SEMÁFORO INTELIGENTE



Asignatura: Estructura de Datos
Profesor: Erwin Michel Davila Iniesta
Alumnos:
Cadena Carrillo Julio César
Correa Martínez Karen Lisette
Correa Yáñez Rubén Jael
Mata Cano Luis Enrique
Yáñez Plata Antonio

Fecha: 23 de abril de 2026



Índice

1. Cinemática y Dinámica	3
1.1. Fundamentos Cinemáticos en Entornos de Tiempo Discreto	3
1.2. Aceleración y Frenado (El Modelo de Impulso)	3
1.3. Fundamentos de Dinámica y la Segunda Ley de Newton	4
1.4. Casos Prácticos de Comportamiento Cinemático	5
2. Calculo Vectorial	6
2.1. Representación Vectorial de la Partícula	6
2.2. Dinámica del Movimiento (Vector Velocidad)	6
2.3. Desarrollo y Ejemplos Prácticos	7
2.4. Análisis de Proximidad (Detección de Colisiones)	8
3. Lenguajes Formales y Autómatas	11
3.1. Diseño del Autómata (Máquina de Moore y AFD)	11
3.2. Lenguaje, Expresión Regular y Gramática Formal	13
3.3. Optimización del autómata.	13
4. Circuitos Eléctricos	14
4.1. El semáforo es un circuito secuencial,	14
4.2. Funcionamiento Lógico (Estados y Entradas)	14
4.3. Funciones de Hardware Específicas.....	16
4.4. Link simulación:	16
4.5. Simulación Tinkercad	17
5.Descripción General de la Simulación.....	18
5.1. Bloque 1: Configuración Principal y Reglas Físicas.....	18
5.2. Bloque 2: Carga de Imágenes y Recursos	19
5.3 Bloque 3: Diseño de los Autos y Semáforos (Plantillas)	20
5.4. Bloque 4: El Cerebro de los Semáforos	22
5.5. Bloque 5: El Bucle Principal (Motor de la Simulación)	22
6. Bibliografías	26

1. Cinemática y Dinámica

Análisis Cinemático y Dinámico: Simulación de un semáforo

Este documento presenta un análisis de ingeniería sobre el simulador bidimensional de tráfico vehicular desarrollado en Python (Pygame). El entorno gráfico controla el flujo vehicular calculando la cinemática de las entidades mediante diferencias finitas (tiempo discreto).

1.1. Fundamentos Cinemáticos en Entornos de Tiempo Discreto

La cinemática describe el movimiento de los cuerpos sin considerar las causas que lo provocan (Stewart, 2017; Thomas, Weir & Hass, 2015). En el código, la simulación de tiempo discreto opera donde cada iteración del bucle principal representa un diferencial de tiempo $dt = 1/60$ s (60 FPS).

▪ Ecuación de Posición y Velocidad

La ecuación diferencial fundamental establece que la velocidad es la derivada de la posición respecto al tiempo ($v = ds/dt$). Despejando para la posición, obtenemos $ds = v \cdot dt$ (Zill & Wright, 2011).

- s : Posición o espacio recorrido del vehículo.
- v : Velocidad del vehículo.
- dt =: Diferencial de tiempo.

En el código fuente, esta integración numérica (método de Euler) se aplica directamente en el método mover(), bajo la instrucción:

```
self.x += self.vel * self.sentido
```

Dado que el bucle es constante, el programa asume implícitamente que $\Delta t = 1$ (un tick del reloj). Matemáticamente, el vector de posición se obtiene integrando la función de velocidad respecto al tiempo:

$$r(t) = r_0 + \int v(\tau) d\tau$$

- $r(t)$: Vector de posición en función del tiempo.
- r_0 : Posición inicial.
- $\int v(\tau) d\tau$: Representa el desplazamiento acumulado.
- $v(\tau)$: Velocidad en función del tiempo dentro de la integral.

1.2. Aceleración y Frenado (El Modelo de Impulso)

La aceleración es el cambio de velocidad en el tiempo ($a = dv/dt$). En la simulación, los vehículos no presentan una aceleración progresiva de arranque, pero sí ejecutan una desaceleración de altísima magnitud durante el frenado.

El código no declara una variable de aceleración explícita, sino que utiliza lógica booleana para dictar el estado cinemático:

self.vel = 0 if bloqueado else self.vel_base

4

Matemáticamente, esto implica que la aceleración actúa como una Función Delta de Dirac. Cuando un vehículo se bloquea en un semáforo rojo, experimenta una deceleración casi infinita en un instante para garantizar la prevención de colisiones sin solapamiento de hitboxes.

1.3. Fundamentos de Dinámica y la Segunda Ley de Newton

Aunque el entorno no programa vectores de fuerza continuos, la Segunda Ley de Newton ($\sum F = m \cdot a$) rige la interacción estructural del frenado. El motor asume un entorno ideal con coeficiente de fricción nulo ($\mu = 0$) (Millington, 2010).

▪ Cálculo de Fuerzas Extremas de Frenado

Para analizar esto con las ecuaciones del movimiento, calculamos la fuerza y aceleración necesarias para llevar un vehículo al reposo absoluto en el lapso de un solo fotograma ($dt = 0.0166$ s).

- **Velocidad inicial:** 80 km/h (equivalente a 22.22 m/s).
- **Aceleración (a):** $(v_f - v_i) / dt = (0 - 22.22) / 0.0166 \approx -1338.55 \text{ m/s}^2$.

Asumiendo una masa promedio de $m = 1500$ kg para un vehículo estándar:

- **Fuerza (F):** $m \cdot a = 1500 \cdot -1338.55 \approx -2,007,825 \text{ N}$.
- **F(Fuerza):** Es la acción que puede cambiar el movimiento de un objeto (empujar o jalar). Se mide en **newtons (N)**.
- **m(masa):** Es la cantidad de materia que tiene un objeto. Indica qué tan difícil es moverlo. Se mide en **kilogramos (kg)**.
- **a(acceleración):** Es el cambio de la velocidad con el tiempo (puede aumentar, disminuir o cambiar de dirección). Se mide en **metros por segundo al cuadrado (m/s²)**.

Aplicar más de dos millones de Newtons por entidad para evitar superposiciones requeriría un costo de procesamiento computacional innecesario, lo que fundamenta por qué el frenado se maneja mediante cambios de estado booleanos (bloqueado = True).

▪ Análisis de las Métricas y Calibración ("Números Mágicos")

El código utiliza valores empíricos para el control del movimiento en pantalla. En la computación gráfica, el "universo" es la cuadrícula bidimensional, siendo el píxel la unidad atómica e indivisible de distancia. La velocidad nativa es píxeles por fotograma (px/frame).

▪ Origen de las Velocidades Base

El sistema genera velocidades con `random.uniform(2.8, 6.4)`. Estos valores son constantes de calibración visual, no nacen de una ley física:

- **2.8 px/frame:** Diseñado para verse "lento pero fluido".
- **6.4 px/frame:** Diseñado para verse "peligrosamente rápido" sin fallar en las cajas de colisión (hitboxes).

▪ Conversión a km/h y Límite del Radar

El radar multa a vehículos sobre 80 km/h. El código usa un factor de escala arbitrario (18):
 $\text{km/h} = (\text{px/frame}) \cdot 18$.

Despejando analíticamente para el límite legal de 80 km/h, se obtiene $80 / 18 = 4.44 \text{ px/frame}$. Cualquier avance superior a esto por fotograma dispara la fotomulta de manera inmediata.

1.4. Casos Prácticos de Comportamiento Cinemático

 Caso Físico	 Aceleración	 Velocidad	 Posición
 A. Crucero (MRU)	$a(t) = 0$ El auto mantiene un movimiento rectilíneo uniforme sin cambios de rapidez.	$v(t) = v_0$ (En código: <code>self.vel_base</code>) La velocidad constante asignada al generar el auto.	$x(t) = x_0 + v_0 \cdot t$ (Bucle de suma)
 B. Frenado de Emergencia	Escalón: $a(t) = -v_{(base)} \cdot \delta(t - t_{(bloqueo)})$ Representa un frenado instantáneo (Escalón) en el momento exacto del bloqueo.	$v(t) = 0 \text{ para } t \geq t_{(bloqueo)}$ La velocidad cae a cero de forma inmediata y permanece así mientras el auto esté detenido.	$x(t) = \text{Constante}$ (Auto detenido): La posición deja de actualizarse, lo que visualmente mantiene al auto detenido en un punto fijo.

2. Calculo Vectorial

El cálculo vectorial es el motor matemático que permite que los vehículos en la simulación se muevan, detecten colisiones y reaccionen al entorno (Stewart, 2017). En el código actual, cada vehículo es esencialmente un **punto material** que se desplaza en un espacio bidimensional $\mathbb{R}^2$.

Se aplicarán conocimientos vistos en la materia para poder realizarlos cálculos correspondientes y poder implementar los conocimientos previos en la simulación.

2.1. Representación Vectorial de la Partícula

Cada vehículo se define mediante un **vector de posición** $r(t)$, cuyas componentes en el plano cartesiano evolucionan en función del tiempo:

$$r(t) = \langle x(t), y(t) \rangle$$

- $r(t)$: (**Vector de Posición**): Es la función vectorial que describe la ubicación exacta del vehículo en el plano cartesiano en cualquier instante de tiempo t .
- $x(t)$: (**Componente Horizontal**): Representa la coordenada en el eje de las abscisas (horizontal). En tu código, esta componente evoluciona según la dirección del flujo vehicular.
- $y(t)$: (**Componente Vertical**): Representa la coordenada en el eje de las ordenadas (vertical).

2.2. Dinámica del Movimiento (Vector Velocidad)

La traslación de los objetos se rige por el vector velocidad v , que define tanto la rapidez (magnitud) como el sentido del flujo vehicular. El desplazamiento incremental en cada frame se calcula mediante la ecuación:

$$r_{n+1} = r_n + v\Delta t$$

- $\vec{r}_{n+1}$ (**Siguiente Posición**): Es el vector que indica dónde estará el vehículo en el próximo fotograma de la simulación.
- $\vec{r}_n$ (**Posición Actual**): Es el vector que indica la ubicación exacta del vehículo en el fotograma presente (el valor actual de *self.x* o *self.y*).
- $\vec{v}$ (**Vector Velocidad**): Es la magnitud y dirección del desplazamiento por unidad de tiempo.
- Δt (**Diferencial de Tiempo**): Es el intervalo de tiempo entre cada actualización del motor de juego. En el código, dado que el bucle es constante (60 FPS), el programa asume implícitamente que $\Delta t = 1$ "tick" de reloj.

▪ Transformación Lineal de Escala

Para la auditoría de tráfico, se aplica una **operación de escalamiento** que transforma la magnitud de velocidad del espacio virtual (píxeles/frame) al sistema métrico real (km/h). Dado que el límite legal establecido es de **80 km/h**, se utiliza la función de transformación:

$$f(v) = \|v\| * 18$$

- $f(v)$ (**Función de Transformación**): Es el resultado final de la conversión, que en tu documento se identifica como la velocidad en km/h.
- $\|v\|$ (**Norma o Módulo del Vector**): Representa la rapidez escalar del vehículo. Es la magnitud del vector velocidad sin tener en cuenta su dirección o sentido. En tu código, este valor corresponde a la *velocidad_base* generada aleatoriamente.
- **18(Factor de Escalamiento)**: Es una constante de calibración o "factor de escala arbitrario". Se utiliza para proyectar los valores del espacio virtual al sistema métrico real.

2.3. Desarrollo y Ejemplos Prácticos

A continuación, se detalla cómo los conceptos anteriores se ejecutan dentro de la lógica del simulador programado en Python.

▪ Magnitud y Auditoría de Velocidad

El sistema genera vehículos con distintas magnitudes de velocidad (Thomas, Weir & Hass, 2015).eu Para determinar si un vehículo es infractor, el programa evalúa el módulo del vector velocidad frente al límite escalar.

- **Fórmula:** $V_{kmh} = \|v\| * 18$
 - V_{kmh} (**Velocidad en Km/h**): Es la magnitud final que el programa muestra en la interfaz gráfica (GUI) para determinar si un conductor es infractor.
 - $\|v\|$ (**Norma o Módulo del Vector**): Representa la rapidez del vehículo (la longitud del vector velocidad) medida en píxeles por fotograma (*px/frame*).
 - **18(Factor de Conversión)**: Es el multiplicador escalar definido en tu código para escalar la velocidad del mundo virtual al sistema métrico real.
- **Procedimiento:** 1. Un auto se genera con una velocidad base de 5.0 px/frame.
2. Se aplica la transformación: $5.0 \cdot 18 = 90$.

- **Resultado:** Como $90 > 80$ (sistema de límite de velocidad), el sistema detecta una infracción y activa el evento de “radar flash”.

2.4. Análisis de Proximidad (Detección de Colisiones)

La prevención de accidentes se basa en la **Norma de la Diferencia** entre dos vectores de posición de vehículos distintos ($\mathbf{r}_{self}$ y $\mathbf{r}_{otro}$) (Zill & Wright, 2011).

- **Fórmula de Distancia Euclidiana:**

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **d (Distancia):** Representa la separación espacial más corta (en línea recta) entre dos vehículos en el plano $\mathbb{R}^2$.
- (x_2, y_2) : Son las coordenadas del (otro vehículo) ($otro.x$, $otro.y$) que circula cerca.
- (x_1, y_1) : Son las coordenadas del (propio vehículo) ($self.x$, $self.y$) que realiza la comprobación.
- $(x_2 - x_1)^2$: Es el cuadrado de la diferencia en el eje horizontal, lo que asegura que el resultado sea siempre positivo.
- $(y_2 - y_1)^2$: Es el cuadrado de la diferencia en el eje vertical.
- $\sqrt{\dots}$ (**Raíz Cuadrada**): Se aplica para obtener la magnitud real de la distancia tras haber sumado los cuadrados de las componentes.
- **Implementación:** En el código, para optimizar el rendimiento, se evalúan las componentes del vector diferencia. Si la distancia en el eje de movimiento es menor a 85 unidades y la desalineación en el eje perpendicular es mínima, el vehículo iguala su magnitud de velocidad a 0 (frenado total).



▪ Integración de Temas Vistos en Clase

La simulación permite visualizar conceptos avanzados de la materia de Cálculo Vectorial que pueden ser integrados para mayor realismo:

- **Campos Escalares:** Las "zonas de radar" y "zonas de cruce" definidas en el código (ZONA_RADAR_X, ZONA_CRUCE_Y) funcionan como regiones de un campo escalar donde se disparan funciones específicas (multas o cambio de estado de cruce) cuando las coordenadas del vector posición entran en dicho rango.
- **Dirección y Sentido:** El código utiliza un **vector unitario de sentido** (± 1) que, al multiplicarse por la magnitud de la velocidad, determina hacia dónde apunta el vector resultante en los ejes **i** (horizontal) o **j** (vertical).
- **Producto Punto (Propuesta):** Podría utilizarse para determinar el campo de visión del conductor, asegurando que solo reaccione a obstáculos que se encuentren en un ángulo positivo respecto a su vector dirección actual.

 Concepto Vectorial	 Aplicación en el Código	 Resultado Operativo
 Vector Posición	Atributos <code>self.x, self.y</code>	 Ubicación precisa en el mapa.
 Escalamiento	<code>velocidad_base : 18</code>	 Conversión a unidades de tráfico real.
 Umbral de Norma	<code>DISTANCIA_FRENADO = 85</code>	 Mantenimiento de la distancia de seguridad.
 Límite Escalar	<code>LIMITE_VELOCIDAD_KMH = 80</code>	 Criterio para el sistema de fotomultas .

A. Producto Punto para Detección de Colisiones

En lugar de usar if para cada eje, podrías usar el producto punto para saber si un vehículo está "frente" a otro.

Si **A** es el vector de dirección del auto y **B** es el vector hacia el otro auto:

$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|}$$

Si el resultado es cercano a **1**, el auto está directamente adelante.

Si es **0**, está a un lado.

- $\cos \theta$ = **(Coseno del ángulo)**: Representa el valor del coseno del ángulo formado entre el vector de dirección del auto y el vector hacia un obstáculo.
- **A: (Vector de Dirección)**: Es el vector unitario que indica hacia dónde está apuntando tu vehículo actúa.
- **B: (Vector hacia el Objetivo)**: Es el vector que nace en tu auto y apunta hacia la posición del otro vehículo o semáforo
- **$\|A\| \|B\|$ (Producto de las Normas)**: Es la multiplicación de las magnitudes (largos) de ambos vectores, utilizada para normalizar el resultado y obtener un valor entre -1 y 1.

B. Vectores de Aceleración y Frenado Suave.

Actualmente, la velocidad cambia de velocidad_base a 0 instantáneamente. Podrías implementar un **Vector de Aceleración a**:

$$v_f = v_i + a \cdot t$$

- v_f **(Velocidad Final)**: Es la rapidez que tendrá el vehículo al terminar la maniobra de aceleración o frenado.
- v_i **(Velocidad Inicial)**: Es la rapidez a la que se desplazaba el auto justo antes de empezar a cambiar su velocidad.
- a **(Aceleración)**: Es el vector de aceleración constante que se aplica al vehículo. Si es positiva, el auto aumenta su velocidad; si es negativa, el auto frena.
- t **(Tiempo)**: Es la duración total del proceso de cambio de velocidad.

Esto haría que los autos no se detengan de golpe, sino que desaceleren suavemente al llegar al semáforo en rojo.

C. Campos de Fuerza (Vectores de Repulsión).

Podrías crear un "campo vectorial" alrededor de cada vehículo. Si un auto entra en el campo de otro, se genera un vector de fuerza opuesta que lo aleja o lo frena, evitando colisiones de forma más orgánica que con una distancia fija.

3. Lenguajes Formales y Autómatas

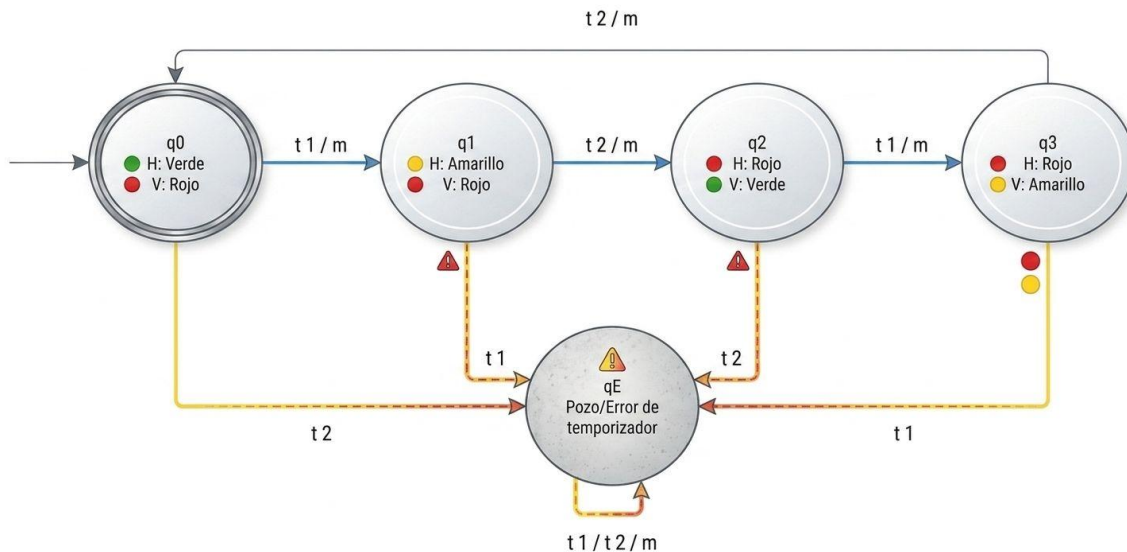
3.1. Diseño del Autómata (Máquina de Moore y AFD)

El sistema se modela como un **Autómata Finito Determinista (AFD)** acoplado a una **Máquina de Moore** (Hopcroft, Motwani & Ullman, 2007). En este modelo, las salidas (los colores de las luces) dependen exclusivamente del estado actual en el que se encuentre el sistema. El autómata se describe formalmente mediante la séxtupla:

$$M = (Q, \Sigma, \Gamma, \delta, \omega, q_0).$$

Definición de Componentes:

- **Conjunto de Estados (Q):** $Q = \{q_0, q_1, q_2, q_3, q_E\}$
 - q_0 : Horizontal Verde / Vertical Rojo.
 - q_1 : Horizontal Amarillo / Vertical Rojo.
 - q_2 : Horizontal Rojo / Vertical Verde.
 - q_3 : Horizontal Rojo / Vertical Amarillo.
 - q_E : Estado Pozo (Error de secuencia de reloj).
- **Alfabeto de Entrada (Σ):** $\Sigma = \{t_1, t_2, m\}$
 - t_1 : Expiración de temporizador largo (8 segundos).
 - t_2 : Expiración de temporizador corto (2 segundos).
 - m : Interrupción manual prioritaria (pulsación de tecla).
- **Alfabeto de Salida (Γ):** Configuración binaria de los 6 focos LED.
- **Función de Salida (ω):** Dicta que la salida física sea encender los LED del semáforo.
- **Estado Inicial (q_0):** q_0 .
- **Conjunto de Estados de Aceptación (F):** $F = \{q_0\}$ (Condición de parada definida como la finalización de un ciclo operativo completo y seguro).



Nota: El sistema inteligente de fotomultas actúa como una función de monitoreo $f(v)$ donde si la velocidad es mayor de 80 km/h, se emite la fotomulta, independientemente del estado actual del autómata.

Análisis de Determinismo:

El autómata es un AFD. Cumple con la propiedad de función de transición total; para cada estado $q \in Q$ y cada símbolo de entrada $a \in \Sigma$, existe exactamente una transición hacia un único estado en Q . No existen bifurcaciones lógicas ante el mismo estímulo.

 Tabla de Transiciones			
Estado Actual	t_1 (Temporizador 8s)	t_2 (Temporizador 2s)	m (Interrupción manual)
q_0	q_1	q_E	q_1
q_1	q_E	q_2	q_2
q_2	q_3	q_E	q_3
q_3	q_E	q_0	q_0
q_E	q_E	q_E	q_E

3.2. Lenguaje, Expresión Regular y Gramática Formal

Lenguaje: El lenguaje aceptado de la maquina son únicamente secuencias que alternan correctamente los ciclos de paso (t_1) y ciclos de precaución (t_2). Permite interrupciones . (m) como sustituto válido de cualquier temporizador, siempre que se respete el orden inmutable de las salidas (los colores del semáforo).

Expresión Regular (ER): Para completar un ciclo válido desde el estado inicial al estado de aceptación (q_0 en ambos casos) sin caer en el estado de error (q_E), la secuencia debe ser:

$$ER = ((t_1 + m)(t_2 + m)(t_1 + m)(t_2 + m))^*$$

Gramática Formal (G):

$$G = (V, \Sigma, R, S)$$

- $V = \{S, A, B, C\}$ (Donde $S = q_0, A = q_1, B = q_2, C = q_3$)
- $\Sigma = \{t_1, t_2, m\}$
- $S = S$

Reglas de Producción (R):

- $S \rightarrow t_1 | mA \mid \epsilon$ (La transición vacía ϵ indica la condición de aceptación).
- $A \rightarrow t_2 B \mid mB$
- $B \rightarrow t_1 C \mid mC$
- $C \rightarrow t_2 S \mid mS$

Las transiciones al estado de error no cuentan en las reglas de producción ya que el pozo no conduce a un estado de transición.

El lenguaje L aceptado por nuestro autómata es regular, por lo que cumple con las propiedades de cerradura (Sipser, 2012). Al aplicar la operación de unión entre el lenguaje que representa el ciclo horizontal y el lenguaje del ciclo vertical, el resultado sigue siendo un lenguaje regular. Asimismo, la intersección de nuestro lenguaje con otro lenguaje regular (como el sistema de fotomultas) permite que el sistema mantenga su naturaleza determinista y segura

3.3. Optimización del autómata.

El diseño de este autómata se encuentra en su estado de minimización absoluta, según el Teorema de Myhill – Nerode (Linz, 2016; Tutorials Point, s.f.). Un lenguaje L es regular si $L \sim (L \sim \text{es una relación sobre cadenas } x \text{ e } y, \text{ donde no hay extensión distinguible para } x \text{ e } y) \text{ tiene un número finito de clases de equivalencia, y si este número es } N, \text{ entonces hay } N \text{ estados en un Autómata Finito Determinista (AFD) mínimo que reconoce } L. \text{ Dado esto, se puede concluir que no existe un estado redundante y no se puede combinar algún estado con otro, por ejemplo, } q_0 \text{ con } q_2 \text{ y } q_1 \text{ con } q_3. \text{ Aunque parezca que cada par de estados cuente con similitudes, el alfabeto de salida (las diferentes combinaciones entre las luces de los semáforos) es distinto para cada uno de los estados.}$

4. Circuitos Eléctricos

4.1. El semáforo es un circuito secuencial, (Brookshear & Brylow, 2015). Ya que su salida (el color de la luz) depende del estado anterior y de una señal de tiempo o reloj. En términos avanzados, se comporta como una **Máquina de Moore**, donde la salida es una función exclusiva del estado actual del sistema. Está basado en un microcontrolador (tipo Arduino). Se clasifica como un sistema de **lógica programada** (Brookshear & Brylow, 2015).

Porque, a diferencia de un circuito de lógica cableada (hecho solo con compuertas), la secuencia de los estados y los tiempos están definidos por el firmware que procesa el autómat.

Se define matemáticamente como un Autómata Finito Determinista (AFD) acoplado a una Máquina de Moore.

- **Determinismo:** Para cada estado y estímulo (tiempo o botón), existe una única transición posible, lo que garantiza seguridad en el tráfico.
- **Máquina de Moore:** Las salidas (luces encendidas) dependen exclusivamente del estado actual en el que se encuentra el sistema, y no de las entradas directamente.
- **AFD (Lógica de control):** Gestiona las transiciones entre estados (por ejemplo, de verde a amarillo) basándose en eventos de entrada como el tiempo o un botón.
- **Máquina de Moore (Lógica de salida):** Establece que las luces encendidas dependen exclusivamente del estado actual en el que se encuentre el sistema, y no de cómo se llegó a él.

4.2. Funcionamiento Lógico (Estados y Entradas)

El sistema opera bajo una estructura de cinco estados principales:

- q_0 : Horizontal Verde / Vertical Rojo.
- q_1 : Horizontal Amarillo / Vertical Rojo.
- q_2 : Horizontal Rojo / Vertical Verde.
- q_3 : Horizontal Rojo / Vertical Amarillo.
- q_E : Estado de Error (Pozo), que se activa si hay un fallo en la secuencia del reloj.

El cambio entre estos estados ocurre por tres tipos de estímulos:

1. t_1 : Expiración de un temporizador largo de 8 segundos (para luces verdes).
2. t_2 : Expiración de un temporizador corto de 2 segundos (para luces amarillas).
3. m : Interrupción manual mediante un pulsador físico.

Para que este circuito funcione correctamente en la realidad, se aplican los siguientes conceptos de electrónica y computación:

15

- **Detección de Flanco de Subida (Edge Detection):** El código utiliza lógica para detectar cuando el botón pasa de LOW a HIGH. Esto es vital para evitar que una pulsación humana (que dura mucho tiempo para el procesador) se cuente como miles de pulsaciones rápidas.
- **Resistencias de Pull-Down:** En la implementación física, se menciona el uso de una resistencia de $10k\Omega$ para el pulsador. Esto asegura que cuando el botón no está presionado, el pin de entrada tenga un voltaje de 0V (tierra) definido, evitando ruido eléctrico.
- **Gestión de Desbordamiento (Overflow):** Se utiliza el tipo de dato unsigned long para la función millis(). Esto es fundamental porque un int normal se llenaría y fallaría a los 32 segundos, mientras que unsigned long permite que el circuito funcione por días sin colapsar.
- **Arquitectura No Bloqueante:** A diferencia de usar delay(), se emplea la resta de tiempos (tiempoActual - tiempoReferencia). Esto permite que el microcontrolador siga monitoreando el botón mientras el tiempo transcurre, cumpliendo con los requisitos de un sistema de tiempo real.
- **Teorema de Myhill-Nerode:** Se aplicó para garantizar que el autómata tiene el número mínimo de estados posibles, asegurando que no haya lógica redundante ni desperdicio de memoria en el microcontrolador.
- **Ley de Ohm ($V = I \cdot R$):** Se aplica para calcular las resistencias necesarias para los LEDs. Sin ellas, el exceso de corriente quemaría los componentes o dañaría los pines del microcontrolador.⁴
- **Leyes de Kirchhoff:** Aplicadas para asegurar que la suma de voltajes en el lazo del LED y la resistencia sea igual al voltaje de la fuente (5V)

4.3. Funciones de Hardware Específicas

El firmware está diseñado para optimizar el uso de recursos:

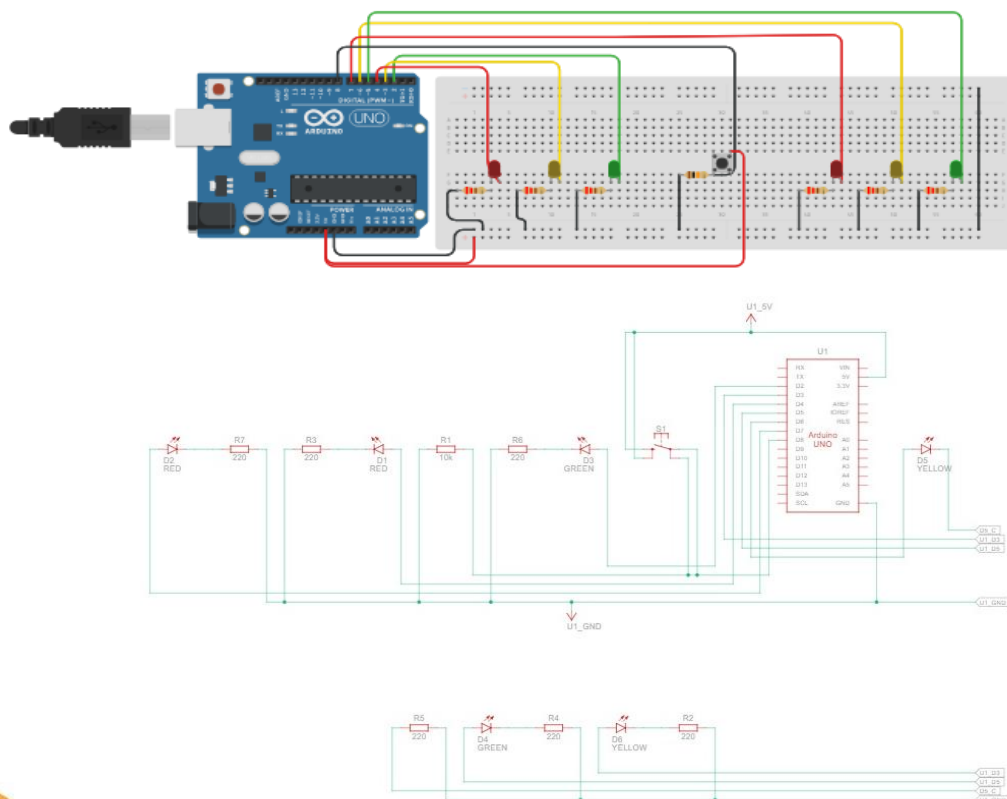
- **Uso de Memoria Flash:** Los pines se declaran como `const int` para que se almacenen en la memoria ROM y no ocupen espacio en la memoria RAM dinámica.
- **Actualización de Salidas:** La función `actualizarLuces()` limpia primero todos los pines (LOW) antes de encender los correspondientes al nuevo estado, garantizando que nunca se queden luces encendidas por error durante la transición.

Fórmulas Relevantes

- **Resistencia para LED:**
$$R = \frac{V_{fuente} - V_{LED}}{I_{led}}$$
- **Temporización:** En el código, el tiempo se gestiona con la función `millis()`, permitiendo ciclos de **8 segundos** (t_1) para verde y **2 segundos** (t_2) para amarillo.

4.4. Link simulación:

<https://www.tinkercad.com/things/kmoNs4pEmmz-circuito-semaforo>



4.5. Simulación Tinkercad

Componentes Principales

- **1 Arduino Uno R3:** El cerebro del proyecto que controla los tiempos y las señales.
- **1 placa de pruebas (Breadboard) grande:** Utilizada para montar todos los componentes y realizar las conexiones de tierra (GND) y voltaje.
- **1 pulsador (Pushbutton):** Ubicado en el centro, probablemente para activar una función de cruce peatonal o cambiar el estado del semáforo.

Dispositivos de Salida (LEDs)

Se observan **6 LEDs** en total, divididos en dos grupos de tres (Rojo, Amarillo, Verde):

- **2 LEDs Rojos.**
- **2 LEDs Amarillos.**
- **2 LEDs Verdes.**

Pasivos (Resistencias)

Se utilizan **7 resistencias** en total para proteger los componentes:

- **6 resistencias para los LEDs:** (Típicamente de **220Ω** a **330Ω**) conectadas al cátodo (pata corta) de cada LED para limitar la corriente.
- **1 resistencia para el pulsador:** (Típicamente de **10kΩ**) configurada como resistencia de *pull-down* para asegurar una lectura digital estable en el pin 2.

Cableado y Conexiones (Jumpers)

En Tinkercad se asignaron colores específicos para mantener el orden, los cuales corresponden a los siguientes pines del Arduino:

- **Cables Negros:** Conexiones a Tierra (**GND**).
- **Cables Rojos:** Conexiones a los LEDs rojos (Pines **13 y 7**) y alimentación positiva del pulsador.
- **Cables Amarillos:** Conexiones a los LEDs amarillos (Pines **12 y 4**).
- **Cables Verdes:** Conexiones a los LEDs verdes (Pines **11 y 3**).
- **Cable Gris:** Conexión de señal del pulsador al pin digital **2**.

5.Descripción General de la Simulación

El programa Metropolis Traffic Control es una simulación gráfica concurrente (Millington, 2010). Desarrollada en Python utilizando la librería Pygame. Su objetivo principal es modelar un flujo vehicular dinámico en una intersección bidimensional controlada por una máquina de estados finitos (los semáforos). Arquitectónicamente, el sistema gestiona múltiples entidades autónomas (vehículos) aplicando abstracciones algorítmicas de cinemática discreta para el movimiento rectilíneo y sistemas de detección geométrica de colisiones. Adicionalmente, el entorno acopla un microservicio concurrente (sistema de fotomulta) que audita la velocidad espacial de los objetos, registrando infracciones en tiempo real y desplegando una interfaz gráfica de usuario (GUI) manipulable por eventos de teclado.

5.1. Bloque 1: Configuración Principal y Reglas Físicas

Este bloque actúa como el "panel de control" del proyecto. Aquí se definen las medidas de la ventana, el límite de velocidad (80 km/h) y las áreas invisibles donde operan los semáforos y el radar. Agrupar estos valores aquí al principio hace que el código sea muy ordenado; si algún día queremos cambiar la distancia de frenado de los autos, solo cambiamos un número aquí arriba en lugar de buscar por todo el programa.

```

1  import pygame
2  import sys
3  import time
4  import random
5  import os
6
7  # --- 1. CONFIGURACIÓN DE ENTORNO Y CONSTANTES DE FÍSICA ---
8  ANCHO_JUEGO, ALTO_JUEGO = 900, 750
9  ANCHO_CONSOLA = 250
10 ANCHO_TOTAL = ANCHO_JUEGO + ANCHO_CONSOLA
11
12 LIMITE_VELOCIDAD_KMH = 80
13 TIEMPO_FLASH_FRAMES = 10
14 DISTANCIA_FRENADO_FRONTAL = 85
15 TOLERANCIA_ALINEACION = 15
16
17 ZONA_RADAR_X = (390, 510)
18 ZONA_RADAR_Y = (380, 470)
19 ZONA_CRUCE_X = (385, 515)
20 ZONA_CRUCE_Y = (375, 475)
21
22 LINEA_FRENO_H_ESTE = (300, 355)
23 LINEA_FRENO_H_OESTE = (540, 595)
24 LINEA_FRENO_V_NORTE = (280, 335)
25 LINEA_FRENO_V_SUR = (465, 520)
26
27 # os.path.abspath(__file__) obtiene la ruta absoluta del archivo en ejecución.
28 # os.path.dirname extrae el directorio padre. Esto garantiza que el programa
29 # encuentre la carpeta 'assets' sin importar desde qué ruta o editor se ejecute.
30 DIRECTORIO_BASE = os.path.dirname(os.path.abspath(__file__))
31
32 pygame.init()
33 pantalla = pygame.display.set_mode((ANCHO_TOTAL, ALTO_JUEGO))
34 pygame.display.set_caption("Metropolis Traffic Control")

```


5.2. Bloque 2: Carga de Imágenes y Recursos

Aquí se preparan los dibujos del asfalto y de los vehículos. El código está diseñado para ser a prueba de errores: busca automáticamente dónde están guardadas las imágenes en la computadora. Si por algún accidente se borra la imagen de un auto, el programa está protegido para no colapsar ni cerrarse; en su lugar, dibujará un cuadro rojo para avisarnos visualmente de que falta ese archivo, permitiendo que la simulación siga corriendo.

19

```
37 # --- 2. GESTIÓN DE RECURSOS ---
38 def cargar_recurso_grafico(nombre_archivo, dimensiones=None):
39     """
40     Carga imágenes en memoria. Utiliza convert_alpha() para que Pygame renderice
41     por hardware el canal de transparencia (los bordes invisibles del PNG),
42     optimizando el rendimiento de fotogramas por segundo (FPS).
43     """
44     ruta_absoluta = os.path.join(DIRECTORIO_BASE, "assets", nombre_archivo)
45     try:
46         imagen = pygame.image.load(ruta_absoluta).convert_alpha()
47         return pygame.transform.scale(imagen, dimensiones) if dimensiones else imagen
48     except FileNotFoundError:
49         print(f"[CRITICAL] Recurso no encontrado: {ruta_absoluta}")
50         superficie_error = pygame.Surface(dimensiones if dimensiones else (50, 50))
51         superficie_error.fill((200, 0, 0))
52         return superficie_error
53
54 IMG_FONDO = cargar_recurso_grafico("image_15.png", (ANCHO_JUEGO, ALTO_JUEGO))
55
56 # Comprensión de Listas (List Comprehension): Construye la lista iterando directamente
57 # sobre el arreglo [0, 2, 3, 4, 5]. Es más eficiente en memoria que un bucle for tradicional.
58 AUTOS_DISPONIBLES = [cargar_recurso_grafico(f"image_{i}.png", (58, 26)) for i in [0, 2, 3, 4, 5]]
```

5.3 Bloque 3: Diseño de los Autos y Semáforos (Plantillas)

En esta sección se crean los "moldes" o plantillas (llamadas clases) para los objetos del cruce:

20

Vehículo: Define cómo nace un auto, si va rápido o lento, hacia dónde apunta y las reglas que debe seguir. Contiene las instrucciones para que el auto sepa cómo frenar si el semáforo está en rojo o si tiene otro auto enfrente. También incluye el código para que el radar detecte su velocidad y le tome una "foto" (multa).

Semáforo: Es un objeto sencillo que guarda su color actual (verde, amarillo o rojo) y lleva un reloj interno para saber exactamente cuánto tiempo lleva encendido.

```

60 # --- 3. CLASES DE DOMINIO ---
61 class Vehiculo:
62     def __init__(self, x, y, eje_movimiento, sentido_vectorial, semaforo_referencia):
63         self.id_vehiculo = random.randint(1000, 9999)
64         self.x = x
65         self.y = y
66         self.eje_movimiento = eje_movimiento
67         self.sentido_vectorial = sentido_vectorial
68         self.semaforo = semaforo_referencia
69
70         self.superficie_grafica = self._generar_sprite()
71         self.ancho, self.alto = self.superficie_grafica.get_size()
72
73         es_infractor = random.random() < 0.20
74         self.velocidad_base = random.uniform(4.6, 6.4) if es_infractor else random.uniform(2.8, 4.4)
75         self.velocidad_actual = self.velocidad_base
76         self.velocidad_kmh = int(self.velocidad_base * 18)
77
78         self.ha_cruzado_interseccion = False
79         self.infraccion_registrada = False
80         self.frames_restantes_flash = 0
81
82     def _generar_sprite(self):
83         imagen_base = random.choice(AUTOS_DISPONIBLES)
84         if self.eje_movimiento == "H":
85             return imagen_base if self.sentido_vectorial == 1 else pygame.transform.flip(imagen_base,
86             return pygame.transform.rotate(imagen_base, 270 if self.sentido_vectorial == 1 else 90)
87
88     def mover(self, flujo_vehicular, registro_multas):
89         esta_bloqueado = self._detectar_colision_frontal(flujo_vehicular)

```

```

91     if not self.ha_cruzado_interseccion and not esta_bloqueado:
92         esta_bloqueado = self._evaluar_frenado_semaforo()
93         self._auditar_velocidad(registro_multas)
94         self._actualizar_estado_cruce()
95
96     self.velocidad_actual = 0 if esta_bloqueado else self.velocidad_base
97
98     if self.eje_movimiento == "H":
99         self.x += self.velocidad_actual * self.sentido_vectorial
100     else:
101         self.y += self.velocidad_actual * self.sentido_vectorial
102
103 def _detectar_colision_frontal(self, flujo_vehicular):
104     """
105     Evalúa colisiones inyectando la lista de todos los vehículos activos.
106     Se utiliza cálculo de distancia Euclidiana simplificada por eje para minimizar el costo del CPU.
107     """
108     for otro in flujo_vehicular:
109         if otro == self:
110             continue
111
112         if self.eje_movimiento == "H":
113             distancia = otro.x - (self.x + self.ancha) if self.sentido_vectorial == 1 else self.x - (otro.x + otro.ancha)
114             desalineacion = abs(self.y - otro.y)
115         else:
116             distancia = otro.y - (self.y + self.alto) if self.sentido_vectorial == 1 else self.y - (otro.y + otro.alto)
117             desalineacion = abs(self.x - otro.x)
118
119         if 0 < distancia < DISTANCIA_FRENADO_FRONTAL and desalineacion < TOLERANCIA_ALINEACION:
120             return True
121     return False

```

```

123 def _evaluar_frenado_semaforo(self):
124     if self.semaforo.estado == "verde":
125         return False
126
127     if self.eje_movimiento == "H":
128         frente = self.x + self.ancha if self.sentido_vectorial == 1 else self.x
129         rango = LINEA_FRENO_H_ESTE if self.sentido_vectorial == 1 else LINEA_FRENO_H_OESTE
130     else:
131         frente = self.y + self.alto if self.sentido_vectorial == 1 else self.y
132         rango = LINEA_FRENO_V_NORTE if self.sentido_vectorial == 1 else LINEA_FRENO_V_SUR
133
134     return rango[0] < frente < rango[1]
135
136 def _auditar_velocidad(self, registro_multas):
137     # Transductor concurrente: Evalúa la posición geométrica contra la matriz central
138     en_zona_radar = (ZONA_RADAR_X[0] < self.x < ZONA_RADAR_X[1]) or (ZONA_RADAR_Y[0] < self.y < ZONA_RADAR_Y[1])
139     if en_zona_radar and self.velocidad_kmh > LIMITE_VELOCIDAD_KMH and not self.infraccion_registrada:
140         self.infraccion_registrada = True
141         self.frames_restantes_flash = TIEMPO_FLASH_FRAMES
142         # Añade un diccionario a la lista compartida de registros
143         registro_multas.append({
144             "id": self.id_vehiculo,
145             "vel": self.velocidad_kmh,
146             "hora": time.strftime("%H:%M")
147         })
148
149 def _actualizar_estado_cruce(self):
150     if (self.eje_movimiento == "H" and ZONA_CRUCE_X[0] < self.x < ZONA_CRUCE_X[1]) or \
151        (self.eje_movimiento == "V" and ZONA_CRUCE_Y[0] < self.y < ZONA_CRUCE_Y[1]):
152         self.ha_cruzado_interseccion = True

```


5.4. Bloque 4: El Cerebro de los Semáforos

Este pequeño bloque contiene la función `alternar_luces`. Es la regla de seguridad del cruce. Se asegura de que los cambios de color se hagan en el orden correcto. Si la calle horizontal pasa a rojo, esta función inmediatamente pone la calle vertical en verde y reinicia los cronómetros. Esto evita accidentes de lógica, asegurando que el programa nunca permita dos luces verdes al mismo tiempo.

```
# --- 4. ORQUESTACIÓN Y CONTROL ---
def alternar_luces(sem_h, sem_v):
    """
    Motor del Autómata Finito Determinista (AFD).
    Modifica la variable de estado y restablece el cronómetro a time.time() actual
    para comenzar a medir el siguiente ciclo en la máquina de estados.
    """
    ahora = time.time()
    if sem_h.estado == "verde":
        sem_h.estado = "amarillo"
    elif sem_h.estado == "amarillo":
        sem_h.estado = "rojo"
        sem_v.estado = "verde"
        sem_v.timer = ahora
    elif sem_v.estado == "verde":
        sem_v.estado = "amarillo"
    elif sem_v.estado == "amarillo":
        sem_v.estado = "rojo"
        sem_h.estado = "verde"
        sem_h.timer = ahora
    sem_h.timer = ahora
```

5.5. Bloque 5: El Bucle Principal (Motor de la Simulación)

Esta es la parte que mantiene al programa vivo y en movimiento, repitiéndose a sí misma 60 veces por segundo para crear la ilusión de video. En cada repetición (fotograma), hace cinco tareas rápidamente:

Revisa el reloj: Checa si ya pasaron los 8 segundos para cambiar la luz de los semáforos automáticamente.

Escucha al usuario: Revisa si presionaste la barra espaciadora para cambiar la luz a mano, o si estás revisando la lista de multas.

Crea tráfico: Decide al azar si debe hacer aparecer un nuevo auto en alguna de las cuatro orillas de la calle.

Mueve y limpia: Avanza los autos hacia adelante. Cuando un auto sale de la pantalla, este bloque lo borra de la memoria de la computadora para que el programa no se vuelva lento con el tiempo.

Dibuja la pantalla: Actualiza los colores, los textos de las multas y los autos para que los veamos moverse.

```

200 # --- 5. BUCLE DE EJECUCIÓN PRINCIPAL ---
201 # Se ajustan Las coordenadas 'Y' y 'X' para colocar Las cajas en Las esquinas de Las aceras
202 semaforo_horizontal = Semaforo(350, 420, "verde")
203 semaforo_vertical = Semaforo(510, 220, "rojo")
204 lista_vehiculos = []
205 registro_multas = []
206 multa_seleccionada = 0
207
208 while True:
209     ahora = time.time()
210
211     # Condición de transición del AFD: si el delta de tiempo supera el límite estático
212     if (semaforo_horizontal.estado == "verde" and ahora - semaforo_horizontal.timer > 8) or \
213         (semaforo_horizontal.estado == "amarillo" and ahora - semaforo_horizontal.timer > 2) or \
214         (semaforo_vertical.estado == "verde" and ahora - semaforo_vertical.timer > 8) or \
215         (semaforo_vertical.estado == "amarillo" and ahora - semaforo_vertical.timer > 2):
216         alternar_luces(semaforo_horizontal, semaforo_vertical)
217
218     pantalla.blit(IMG_FONDO, (0, 0))
219
220     for evento in pygame.event.get():
221         if evento.type == pygame.QUIT:
222             pygame.quit()
223             sys.exit()
224         if evento.type == pygame.KEYDOWN:
225             if evento.key == pygame.K_SPACE:
226                 alternar_luces(semaforo_horizontal, semaforo_vertical)

```



```

228         if registro_multas:
229             if evento.key == pygame.K_UP:
230                 multa_seleccionada = max(0, multa_seleccionada - 1)
231             if evento.key == pygame.K_DOWN:
232                 multa_seleccionada = min(len(registro_multas) - 1, multa_seleccionada + 1)
233             if evento.key in [pygame.K_m, pygame.K_d]:
234                 registro_multas.pop(multa_seleccionada)
235                 multa_seleccionada = 0
236
237     if random.random() < 0.03 and len(lista_vehiculos) < 14:
238         origen = random.randint(1, 4)
239         nodos_generacion = {
240             1: (-100, 373, "H", 1, semaforo_horizontal),
241             2: (1000, 333, "H", -1, semaforo_horizontal),
242             3: (465, -100, "V", 1, semaforo_vertical),
243             4: (412, 850, "V", -1, semaforo_vertical)
244         }
245         # Desempaquetado de argumentos (*): El asterisco extrae los 5 elementos de la tupla
246         # seleccionada en el diccionario y los inyecta como parámetros individuales en __init__
247         lista_vehiculos.append(Vehiculo(*nodos_generacion[origen]))
248
249     # Se usa [:] para iterar sobre una copia de la lista. Esto previene errores de
250     # desplazamiento de índices al eliminar elementos (v) dentro del mismo bucle.
251     for vehiculo in lista_vehiculos[:]:
252         vehiculo.mover(lista_vehiculos, registro_multas)
253         vehiculo.dibujar(pantalla)
254         # Garbage Collection (Liberación de memoria): Si el vehículo sale de los límites
255         # matemáticos de la pantalla, se destruye su instancia para no sobrecargar la RAM.
256         if vehiculo.x > 1100 or vehiculo.x < -200 or vehiculo.y > 950 or vehiculo.y < -200:
257             lista_vehiculos.remove(vehiculo)

```

```

259 semaforo_horizontal.dibujar(pantalla)
260 semaforo_vertical.dibujar(pantalla)
261
262 # --- RENDERIZADO DE INTERFAZ ---
263 pygame.draw.rect(pantalla, (15, 15, 20), (ANCHO_JUEGO, 0, ANCHO_CONSOLA, ALTO_JUEGO))
264 fuente_titulo = pygame.font.SysFont("Impact", 22)
265 fuente_info = pygame.font.SysFont("Arial", 14)
266
267 pantalla.blit(fuente_titulo.render("TRÁNSITO: PENDIENTES", True, (255, 200, 0)), (ANCHO_JUEGO + 10, 30))
268
269 if not registro_multas:
270     pantalla.blit(fuente_info.render("Sin infracciones", True, (100, 100, 100)), (ANCHO_JUEGO + 10, 70))
271 else:
272     # enumerate() permite obtener simultáneamente el índice (0, 1, 2...) y el valor
273     # del elemento iterado, útil para multiplicar y generar espaciado en la interfaz.
274     for indice, multa in enumerate(registro_multas[:15]):
275         color_texto = (0, 255, 255) if indice == multa_seleccionada else (255, 255, 255)
276         cursor = ">" if indice == multa_seleccionada else " "
277         texto = f"{cursor} ID:{multa['id']} | {multa['vel']} km/h"
278         pantalla.blit(fuente_info.render(texto, True, color_texto), (ANCHO_JUEGO + 10, 70 + indice * 22))
279
280 instrucciones = ["[ESPACIO] Cambio Manual", "[ARRIBA/ABAJO] Navegar", "[M] Multar [D] Descartar"]
281 for idx, texto in enumerate(instrucciones):
282     pantalla.blit(fuente_info.render(texto, True, (180, 180, 180)), (ANCHO_JUEGO + 10, 620 + idx * 25))
283
284 pygame.display.flip()
285 reloj.tick(60)

```

6. Bibliografías

1. Brookshear, J. G., & Brylow, D. (2015). *Computer science: An overview* (12th ed.). Pearson.
2. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.
3. Linz, P. (2016). *An introduction to formal languages and automata* (6th ed.). Jones & Bartlett Learning.
4. Millington, I. (2010). *Game physics engine development: How to build a robust commercial-grade physics engine for your game* (2nd ed.). CRC Press.
5. Sipser, M. (2012). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.
6. Stewart, J. (2017). *Cálculo de varias variables: Trascendentes tempranas* (8.^a ed.). Cengage Learning.
7. Thomas, G. B., Weir, M. D., & Hass, J. (2015). *Cálculo: Varias variables* (13.^a ed.). Pearson Educación.
8. Tutorials Point. (s.f.). *Myhill-Nerode theorem in automata*. Recuperado el 18 de abril de 2026, de https://www.tutorialspoint.com/automata_theory/myhill_nerode_theorem.htm
9. Zill, D. G., & Wright, W. S. (2011). *Cálculo de varias variables* (4.^a ed.). McGraw-Hill Interamericana.